PROGINEER · BACKEND TRAINING · RESEARCH REPORT

Pagination for the Donation Management API

Izzaldeen · October 2026 · ASP.NET Core (.NET 10) · EF Core 10 · SQL Server · implemented in **DonationApi** (GET /api/donations)

Decision: keyset (cursor) pagination on `(CreatedAt DESC, Id DESC)` with an opaque cursor, optional filters (date range, minimum amount) and an opt-in total count. It is the only option that stayed fast on deep pages and returned no duplicates when donations were inserted while a user was paging.

1 The three approaches

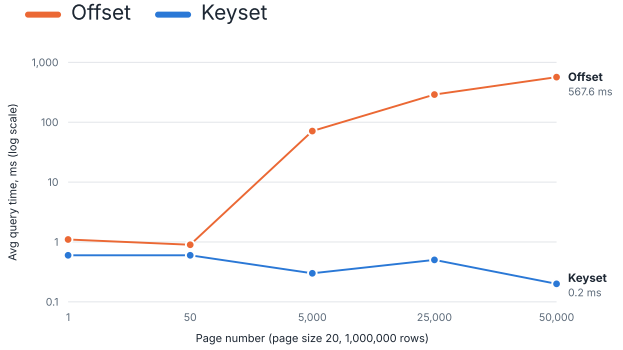
Approach	How it works (EF Core → SQL)	Strengths	Weaknesses
Offset `offset=40 ?` `offset=40&limit=20 amp;limit=20`	`Skip(40).Take(20)` → `OFFSET @p ROWS FETCH NEXT @p1 ROWS ONLY`. The server reads and discards the first N rows.	Simplest; jump to any position; native in EF.	Cost grows with N; inserts/deletes shift rows → duplicates or skipped rows; needs SQL Server 2012+.
Page-based + total `page=3 ?` `page=3&pageSize=20 amp;pageSize=20`	Same OFFSET query with `skip=(page−1)×size`, plus `SELECT COUNT(*)` on every request.	"Page 3 of 50" UI, page links, total size.	All offset problems + a COUNT per request (108 ms on 1M rows); total can be stale.
Keyset / cursor `cursor=eyJ0Ijo…`	`Where(CreatedAt < c OR (CreatedAt = c AND Id < id)).Take(20)` → `SELECT TOP(@p) … WHERE … ORDER BY CreatedAt DESC, Id DESC` (index seek).	Same cost on every page; stable under inserts/deletes; works on any SQL Server.	No jump to page N; no total by itself; each sort needs its own index + cursor.

2 Evidence: 1,000,000 donations, page size 20

Lab copy of the `Donations` table (1M rows, pairs sharing the same `CreatedAt` to test ties), same index; each query run 10× on a warm cache.

Offset grows page after page; keyset stays below 1 ms

Average time per query, log scale (each grid line = 10×).



Page (rows skipped)	Offset ms · reads	Keyset ms · reads
1 (0)	1.1 · 61	0.6 · 61
5,000 (99,980)	71.3 · 753	0.3 · 61
25,000 (499,980)	289.9 · 3,538	0.5 · 61
50,000 (999,980)	567.6 · 6,992	0.2 · 61
Plan, last page	Index Scan, 1,000,000 rows	Index Seek, 20 rows
Through EF Core	157.4 ms	1.4 ms
`COUNT(*)` (total)	107.9 ms · 6,939 reads, on every request	

Real-time inserts: load page 1 → 3 donations arrive → load page 2	
Offset / page-based	3 rows of page 1 shown again
Keyset	0 duplicates, 0 skipped

3 Decision and justification

- **Scale:** keyset reads only *pageSize* rows on any page, so latency does not grow with the data; offset reached 567.6 ms and a full index scan.
- **Data integrity:** donations are inserted continuously; keyset continues strictly after the last row the user saw, so pages never shift.
- **Database efficiency:** one covering index serves the order, the cursor seek and the date filter, with no key lookups.
- **Access pattern:** donations are a time-ordered feed ("newest → next"); users narrow with filters, they do not jump to page 37,412.

Trade-off	Mitigation
No "jump to page N"	Filters `from` / `to` / `minAmount`; first page = no cursor; previous page = client keeps its cursors.
No total by default	`includeTotal=true` runs one COUNT with the same filters only when the UI needs it (can be cached).
Equal timestamps	`Id` is the tie-breaker in the ORDER BY and in the cursor → strict, deterministic order.
One index per sort	Sort is a whitelist (`newest`, `oldest`: same index read both ways). Sorting by amount would add an `(Amount, Id)` index.

When I would choose differently: a small, rarely changing admin table that must show "page X of Y" → **page-based + total**; a quick internal tool with static data → **offset**.

4 Implementation in DonationApi

Part	What was built
API & request DTO	`GET /api/donations?pageSize=20&cursor=…&sort=newest&from=…&to=…&minAmount=…&includeTotal=false` → `DonationPageQuery`: `pageSize` 1–100 (default 20), sort whitelist, `minAmount` > 0, `from ≤ to`, cursor valid and created for the same sort; otherwise **400 ProblemDetails**.
Response	`{ items, pageSize, hasNextPage, nextCursor, totalCount }`; `nextCursor` = Base64Url of `(CreatedAt ticks, Id, sort)`, `null` on the last page.
Index & query	Migration `KeysetPaginationIndex`: `IX_Donations_CreatedAt_Id (CreatedAt DESC, Id DESC) INCLUDE (Amount, DonorName, IsAnonymous)`. One SQL query per page, `AsNoTracking`, projection in SQL (no N+1), `Take(pageSize + 1)` to detect the next page without a COUNT.
Database	The dev database is SQL Server 2008 (no OFFSET/FETCH, so Skip/Take would fail); keyset needs only TOP + WHERE and runs there unchanged.

```
var newest = q.Sort == DonationSort.Newest;
// filters (from, to, minAmount) are applied before this
if (DonationCursor.TryDecode(q.Cursor, out var c))
    query = newest
        ? query.Where(d => d.CreatedAt < c.CreatedAt || (d.CreatedAt == c.CreatedAt && d.Id < c.Id))
        : query.Where(d => d.CreatedAt > c.CreatedAt || (d.CreatedAt == c.CreatedAt && d.Id > c.Id));
query = newest ? query.OrderByDescending(d => d.CreatedAt).ThenByDescending(d => d.Id)
               : query.OrderBy(d => d.CreatedAt).ThenBy(d => d.Id);
var items = await query.Take(q.PageSize + 1).Select(ToResponseExpression).ToListAsync(ct);
var hasNext = items.Count > q.PageSize;  if (hasNext) items.RemoveAt(items.Count - 1);
```

EF Core translates this to one `SELECT TOP(@p) … WHERE [CreatedAt] < @lastCreatedAt OR (… AND [Id] < @lastId) ORDER BY [CreatedAt] DESC, [Id] DESC` query, with the anonymous name masked by a `CASE` inside SQL.

Verification

Test (DonationApi, real database)	Result
Walk all pages newest & oldest; filters + `includeTotal`; anonymous rows	**every row exactly once, correct (CreatedAt, Id) order, totalCount = rows walked, names masked**
3 donations created between page 1 and page 2	**0 duplicates**
pageSize 0 / 101 / abc, bad cursor, cursor of another sort, from > to, unknown sort, minAmount 0	**400 with the field named**
Totals	**26/26** scripted API checks · Postman: **27 requests / 67 tests** (new "Pagination" folder)